Introduction

- Learning the fundamentals about analyzing algorithms in terms of efficiency

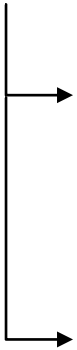
measuring performance (complexity)



complexity analysis

Introduction

- Measures



Time complexity

Amount of time algorithm takes to execute

Space complexity

Amount of space used by algorithm

  - Which is more important?
    &
    Why?

Analysis Framework

- Step 1
  - Input size

    Decide on parameter(s) indicating input size
- Step 2
  - Basic operation

    Identify basic operation
- Step 3
  - Count

    Count how many times basic operation is performed

Most important step

- Step 4
  - Closed-form

    Convert to a closed-form expression
- Step 5
  - Asymptotic notation

    Express in terms of asymptotic notation

Analysis Framework – Input Size

- Time complexity ~ input size
  - Sorting algorithm
    - Sorting 10 items
    - Sorting 1,000 items



f(n)

  - In general:
    - f(n)
    - f(n,m) ⟹ Any example?
    - etc.

Analysis Framework – Basic Operation

- How to measure time complexity?
  - Given *n* inputs, this algorithm executes in 10 msec ???
  - Given *n* inputs, this algorithm executes *n* additions, n^2 multiplications, *5*n* output operations, n^2 function calls, … ???

unnecessarily complex

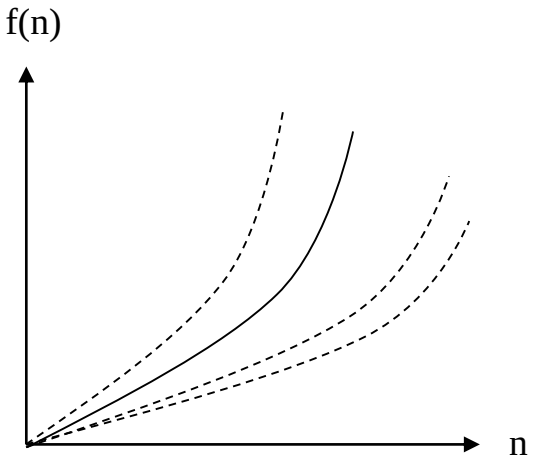
  - A "single" basic operation



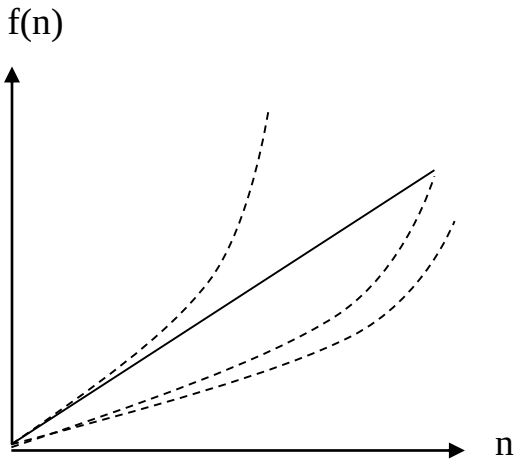
most important operation
algorithm performs repeatedly
contributes most to the total execution time

Analysis Framework – Basic Operation

- Realistic basic operation
  - Analyzed complexity is bounded by constant multiples of the actual execution time



realistic



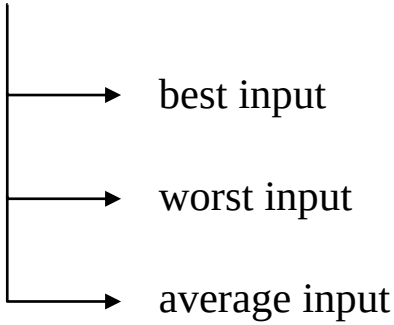
not realistic

Analysis Framework – Counting

- Algorithm analysis

Counting

- Counting



Analysis Framework – Closed-form

- Summation or recurrence



closed-form expression

  - Example:

$f(n) = \sum_{i=1}^{n} i$ → $f(n) = \frac{n(n+1)}{2}$

$f(n) = 2f(n-1) + 1$ → $f(n) = 2^n - 1$

Analysis Framework – Asymptotic Notation

- Simplify more
  - Example:

$$f(n) = \sum_{i=1}^{n} i \rightarrow f(n) = \frac{n(n+1)}{2} \rightarrow f(n) \in \theta(n^2)$$

$$f(n) = 2f(n-1) + 1 \rightarrow f(n) = 2^n - 1 \rightarrow f(n) \in O(2^n)$$

B(n), A(n), W(n)

- Let
  - T_n : set of all inputs of size n
  - $\tau(I)$: number of basic operations for input I

- Best-case complexity
  - $B(n) = min\{\tau(I) \mid I \in T_n\}$

efficiency for the best-case input
input for which the algorithm runs fastest

B(n), A(n), W(n)

- Let
  - T_n : set of all inputs of size n
  - $\tau(I)$: number of basic operations for input I

- Worst-case complexity
  - $W(n) = max\{\tau(I) \mid I \in T_n\}$

efficiency for the worst-case input
input for which the algorithm runs longest

B(n), A(n), W(n)

- Let
  - T_n : set of all inputs of size n
  - $\tau(I)$: number of basic operations for input I

- Average complexity
  - $p(I)$: probability of input I
  - p_k : probability algorithm performs k basic operations ($= \sum_{\tau(I)=k} p(I)$)

$$\Rightarrow A(n) = \sum_{I \in T_n} \tau(I) * p(I)$$



expected value of a random variable (τ)

$$A(n) = \sum_{k=B(n)}^{W(n)} k * p_k$$

efficiency when executed repeatedly
typical behavior

B(n), A(n), W(n)

- Show that

$$A(n) = \sum_{I \in \mathrm{T}_n} \tau(I) * p(I)$$
$$A(n) = \sum_{k=B(n)}^{W(n)} k * p_k$$

} equivalent

$$A(n) = \sum_{I \in \mathrm{T}_n} \tau(I) * p(I)$$
$$= \tau(I_1) * p(I_1) + \tau(I_2) * p(I_2) + \tau(I_3) * p(I_3) + \cdots$$
$$= 1 * \underbrace{\left[p(I_{1,1}) + p(I_{1,2}) + \cdots\right]}_{\sum_{\tau(I)=1} p(I)} + 2 * \underbrace{\left[p(I_{2,1}) + p(I_{2,2}) + \cdots\right]}_{\sum_{\tau(I)=2} p(I)} + \cdots$$

$\sum_{\tau(I)=1} p(I) \rightarrow p_1$

$\sum_{\tau(I)=2} p(I) \rightarrow p_2$

$$= \sum_{k=B(n)}^{W(n)} k * p_k$$

B(n), A(n), W(n)

- Example:

Input	Probability	Basic operations
I_1	0.20	5
I_2	0.40	4
I_3	0.10	5
I_4	0.20	7
I_5	0.10	4

$$\Rightarrow A(n) = \sum_{I \in T_n} \tau(I) * p(I)$$
$$= (5 * 0.20) + (4 * 0.40) + (5 * 0.10) + (7 * 0.20) + (4 * 0.10)$$
$$= 4.90$$

$$A(n) = \sum_{i=B(n)}^{W(n)} i * p_i$$
$$= \big(4 * (0.40 + 0.10)\big) + \big(5 * (0.20 + 0.10)\big) + \big(6 * (0)\big) + \big(7 * (0.20)\big)$$
$$= 4.90$$

B(n), A(n), W(n)

- Comments:
  - Which one(s) are more important to know?
    &
    Why?
  - Which one(s) are more difficult to compute?
    &
    Why?

B(n), A(n), W(n)

- Computing A(n):
  - Determine a probability distribution for the input
    Typical: uniform distribution
  - Calculate the average number of operations

Do not forget:
Calculating A(n) always involves a probability distribution

B(n), A(n), W(n)

- Computing A(n):
  - Example:

Input	Probability	Basic operations
I_1	0.20	5
I_2	0.20	4
I_3	0.20	5
I_4	0.20	7
I_5	0.20	4

uniform distribution

Input	Probability	Basic operations
I_1	0.20	5
I_2	0.40	4
I_3	0.10	5
I_4	0.20	7
I_5	0.10	4

another distribution

  - Example: Quick sort

    uniform distribution vs. nearly sorted input

Asymptotic Analysis

- Converted to closed-form:
  - Example:

$$A(n) = 3n^2 + \frac{100 \log n}{\sqrt{n}} \longrightarrow A(n) \approx n^2$$

$$A(n) = \frac{10+2(n!-5n)}{n^{3/2}} \longrightarrow A(n) \approx n!$$

What is the behavior of this algorithm?

or

Which algorithm is better?



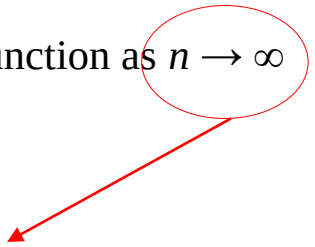
Difficult to observe



Explicit

Asymptotic Analysis

- Principle 1:
  - Look at the growth of the complexity function as $n \rightarrow \infty$

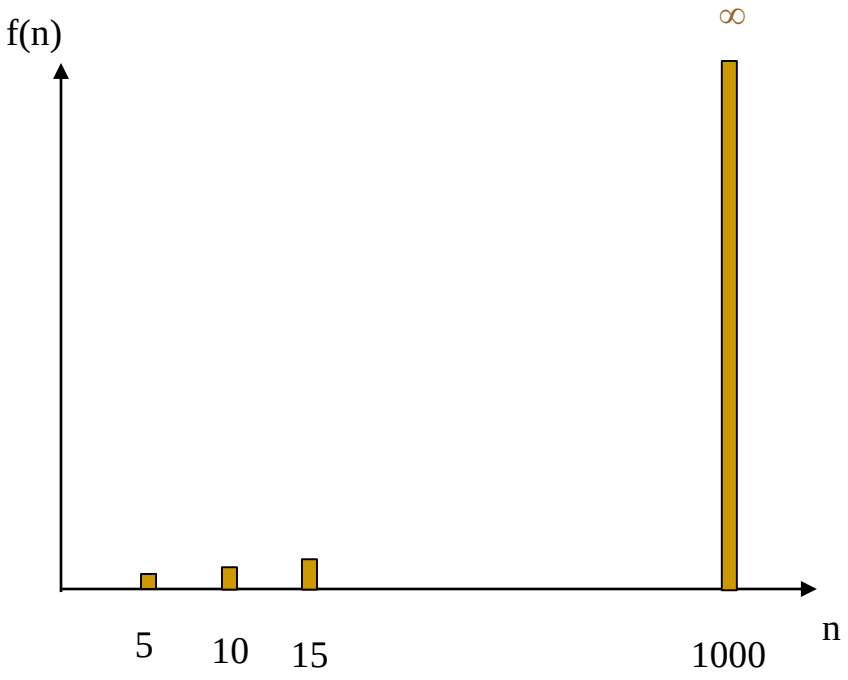


asymptotic analysis

Analyzing the behavior of an algorithm as the input size tends to infinity

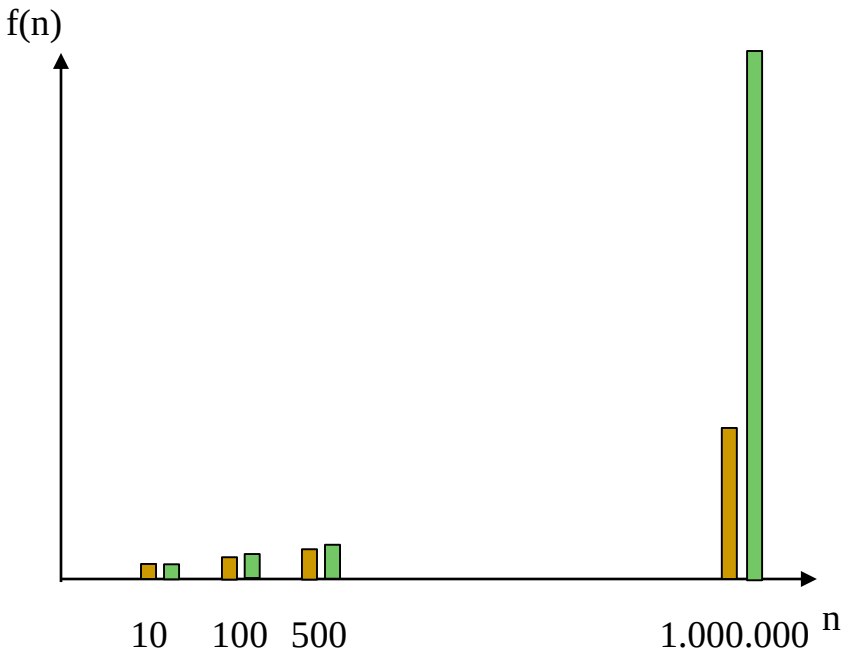
Asymptotic Analysis

- Principle 1:
  - Why asymptotic analysis?
  - Example: Towers of Hanoi algorithm



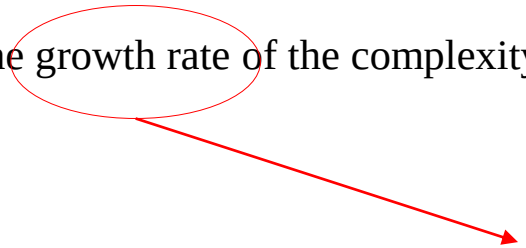
Asymptotic Analysis

- Principle 1:
  - Why asymptotic analysis?
  - Example: Quicksort vs. Bubble sort



Asymptotic Analysis

- Principle 2:
  - Look at the growth rate of the complexity function rather than actual values



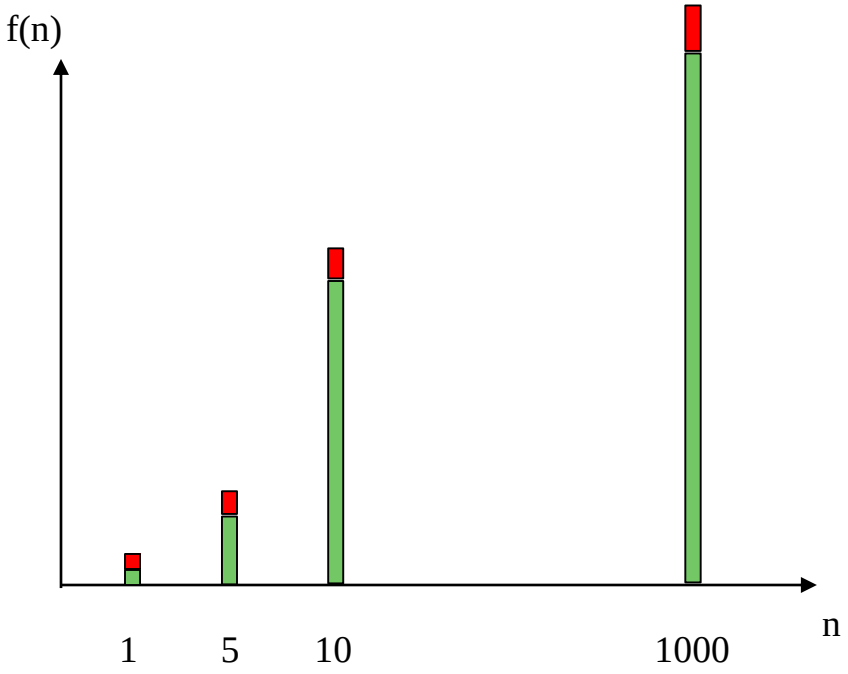
The rate of increase in the function as the input size increases

Asymptotic Analysis

- Low-order terms ✗

  terms other than the fastest growing term

  - Example: $f(n) = n^2 + n$ ⟶ $f(n) \approx n^2$

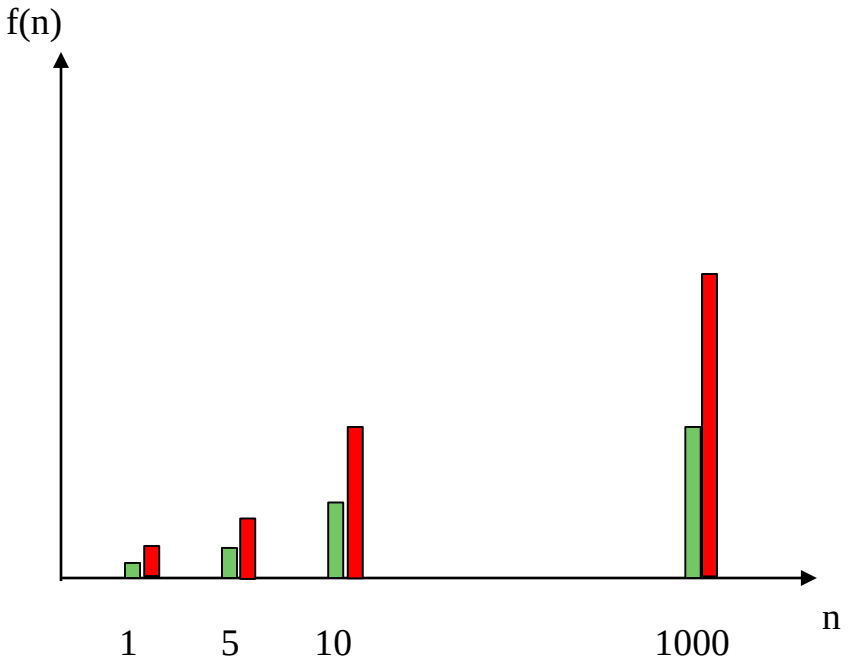


Asymptotic Analysis

- Constant factors 

  constants in the equation

  - Example: $f(n) = 2n^2$ ⟶ $f(n) \approx n^2$

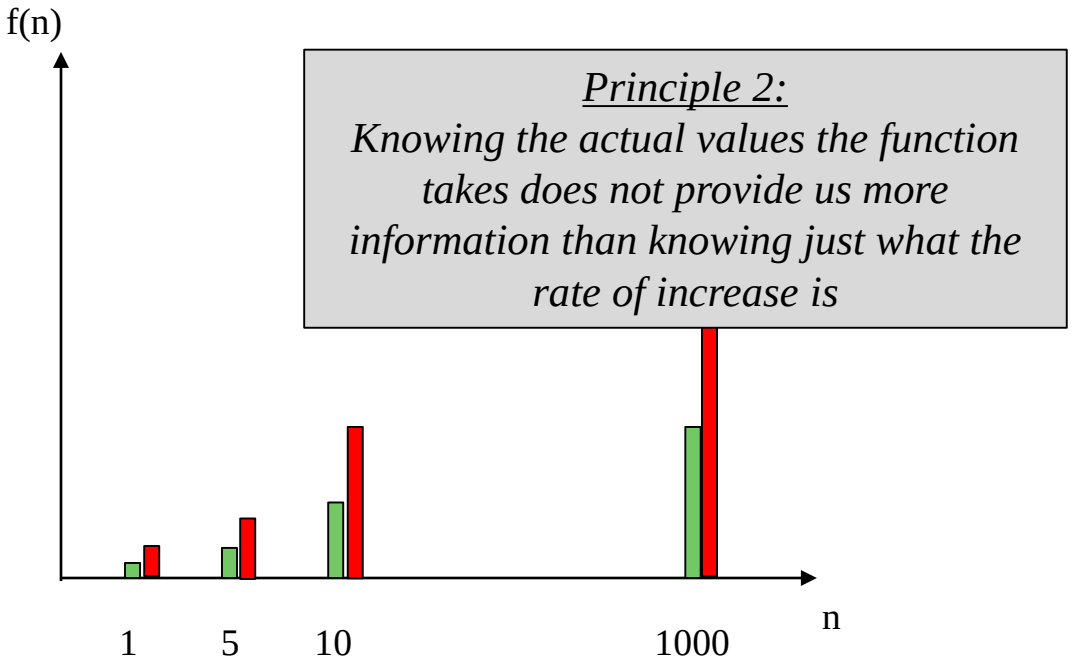


Asymptotic Analysis

- Constant factors 

  constants in the equation

  - Example: $f(n) = 2n^2$ ⟶ $f(n) \approx n^2$



Principle 2:
Knowing the actual values the function takes does not provide us more information than knowing just what the rate of increase is

Asymptotic Notations

- $\mathcal{F}$: set of functions $f(n)$: $\mathbb{N} \rightarrow \mathbb{R}$ that are eventually positive

there exists an n_0 such that $f(n)>0$ for all $n>n_0$

$\mathbb{N}$: input size

$\mathbb{R}$: number of basic operations $\Longrightarrow$ Why $\mathbb{R}$?

- Let $f(n), g(n) \in \mathcal{F}$

Asymptotic Notations

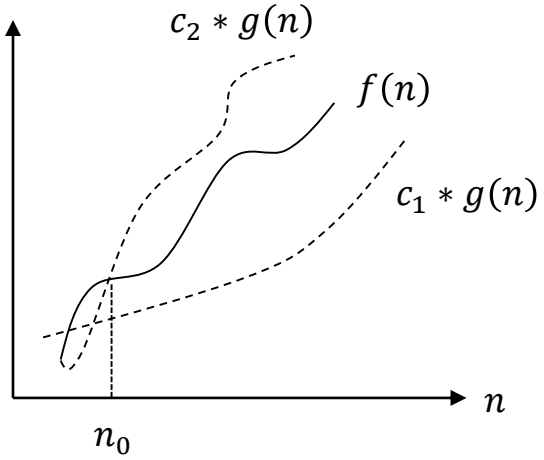
- θ – theta
  - $f(n) \in \theta(g(n))$ iff there exist positive constants c_1, c_2, and n_0 such that $c_1 * g(n) \leq f(n) \leq c_2 * g(n)$, whenever $n \geq n_0$
  - $\theta(g(n))$ set of functions

$f(n) \in \theta(g(n))$ ✓

$f(n) = \theta(g(n))$ ✗

Asymptotic Notations

- θ – theta



$f(n)$ is bounded below and above by some constant multiples of $g(n)$

$f(n)$ grows at the same rate as $g(n)$

Asymptotic Notations

- θ – theta
  - $\theta(g(n))$ → representative of a set of many functions

    Example:

    $\theta(n^2) = \theta(5n^2) = \theta\left({}^{n^2}/_{2} + 2\log n\right) = \cdots$

  - $f(n)$: complexity of an algorithm

    $g(n)$: its complexity class

    $f(n) \in \theta(n^2)$ ✔

    $f(n) \in \theta(5n^2)$ ✘

    $f(n) \in \theta\left({}^{n^2}/_{2} + 2\log n\right)$ ✘

Asymptotic Notations

- θ – theta
  - Example: $n^2 \in \theta(5n^2 + n)$

    $c_1 * (5n^2 + n) \leq n^2 \leq c_2 * (5n^2 + n)$, for $n \geq n_0$

    $c_1 = 1/10$, $c_2 = 1$, $n_0 = 1$ ✔

  - Example: $5n^2 + n \in \theta(n^2)$

    If $f(n) \in \theta(g(n))$, then $g(n) \in \theta(f(n))$

    $$f(n) \in \theta(g(n)) \Rightarrow \underbrace{c_1 * g(n) \leq f(n)}_{g(n) \leq \frac{1}{c_1} * f(n)} \leq \underbrace{c_2 * g(n)}_{\frac{1}{c_2} * f(n) \leq g(n)} \text{ , for } n \geq n_0$$

    $$\Rightarrow \frac{1}{c_2} * f(n) \leq g(n) \leq \frac{1}{c_1} * f(n) \text{ , for } n \geq n_0$$

Asymptotic Notations

- θ – theta
  - Example: $n^2 \notin \theta(n^3)$

    $c_1 * n^3 \leq n^2 \leq c_2 * n^3$, for $n \geq n_0$

    $c_1 * n^3 \leq n^2 \Rightarrow c_1 * n \leq 1$, for $n \geq n_0$

    As $n \to \infty$, $c_1 \to 0 \Rightarrow$ No $c_1 > 0$ 

Asymptotic Notations

- θ – theta
  - Equivalence relation on $\mathcal{F}$

For $f(n), g(n), h(n) \in \mathcal{F}$:

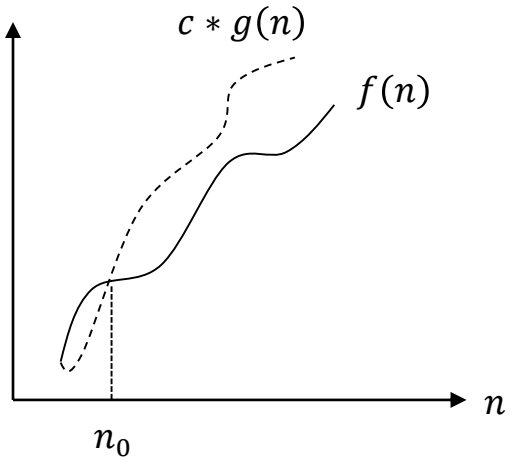
- Reflexivity : $f(n) \in \theta\big(f(n)\big)$
- Symmetricity : $f(n) \in \theta\big(g(n)\big) \Longrightarrow g(n) \in \theta\big(f(n)\big)$
- Transitivity : $f(n) \in \theta\big(g(n)\big)\ and\ g(n) \in \theta\big(h(n)\big) \Longrightarrow f(n) \in \theta\big(h(n)\big)$

Asymptotic Notations

- O – big oh
  - $f(n) \in O(g(n))$ iff there exist positive constants c and n_0 such that $f(n) \leq c * g(n)$, whenever $n \geq n_0$

Asymptotic Notations

- O – big oh



$f(n)$ is bounded above by some constant multiple of $g(n)$

$f(n)$ grows at the same rate as or slower than $g(n)$

O serves as an upper bound

Example: $f(n) \in O(n^2) \Rightarrow$???

Asymptotic Notations

- O – big oh
  - Example: $5n^2 \in O(n^2)$

    $5n^2 \leq c * n^2$, for $n \geq n_0$

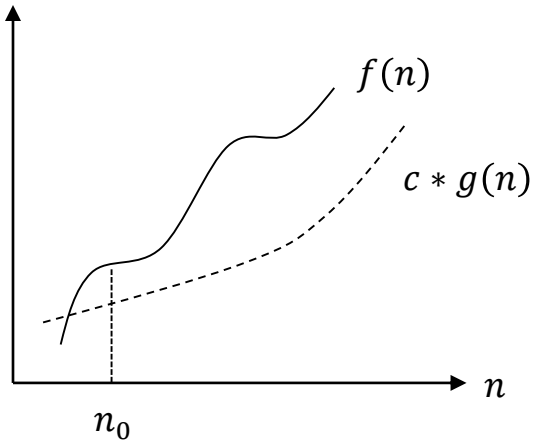
    $c = 10$, $n_0 = 1$ 
  - Example: $n^2 \in O(5n^2)$
  - Example: $n^2 \in O(n^3)$
  - Example: $n^3 \notin O(n^2)$

Asymptotic Notations

- Ω – omega
  - $f(n) \in \Omega(g(n))$ iff there exist positive constants c and n_0 such that $c * g(n) \leq f(n)$, whenever $n \geq n_0$

Asymptotic Notations

- Ω – omega



$f(n)$ is bounded below by some constant multiple of $g(n)$

$f(n)$ grows at the same rate as or faster than $g(n)$

Ω serves as a lower bound

Example: $f(n) \in \Omega(n^2) \Rightarrow$???

→ Useful?

Asymptotic Notations

- Ω – omega
  - Example: $5n^2 \in \Omega(n^2)$

    $c * n^2 \leq 5n^2$, for $n \geq n_0$

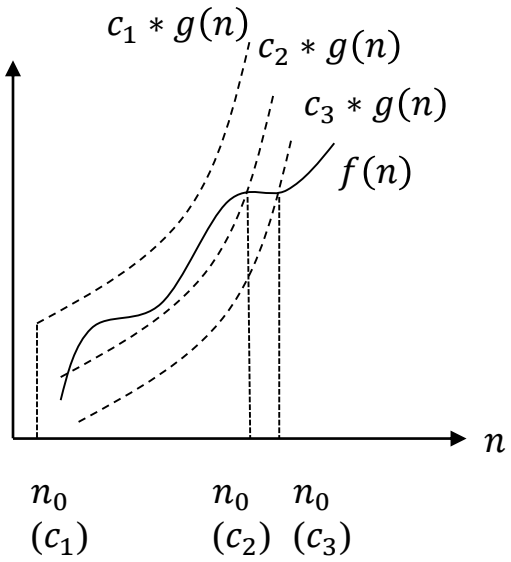
    $c = 1$, $n_0 = 1$ 
  - Example: $n^2 \in \Omega(5n^2)$
  - Example: $n^3 \in \Omega(n^2)$
  - Example: $n^2 \notin \Omega(n^3)$

Asymptotic Notations

- o – little oh
  - $f(n) \in o(g(n))$ iff for every positive constant c, there is a positive integer n_0 such that $f(n) < c * g(n)$, whenever $n \geq n_0$

Asymptotic Notations

- o – little oh



$f(n)$ grows slower than $g(n)$

more strict estimate than O

Asymptotic Notations

- o – little oh
  - Example: $n^2 \in o(n^3)$

    For any $c > 0$, $n^2 < c * n^3$, for $n \geq n_0$
    $\Rightarrow 1 < c * n$
    For any $c > 0$, $n_0 > 1/c$ ✔

  - Example: $n^2 \notin o(5n^2)$

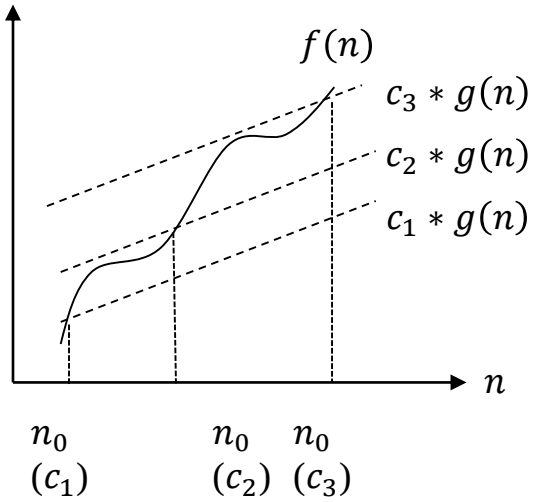
    For any $c > 0$, $n^2 < 5 * c * n^2$, for $n \geq n_0$
    $\Rightarrow 1 < 5 * c$
    $c = 1/10 \Rightarrow 1 < 1/2$ ✘

Asymptotic Notations

- ω – little omega
  - $f(n) \in \omega(g(n))$ iff for every positive constant c, there is a positive integer n_0 such that $c * g(n) < f(n)$, whenever $n \geq n_0$

Asymptotic Notations

- ω – little omega



$f(n)$ grows faster than $g(n)$

more strict estimate than Ω

Asymptotic Notations

- ω – little omega
  - Example: $n^3 \in \omega(n^2)$
  - Example: $n^2 \notin \omega(5n^2)$

Asymptotic Notations

- Informally

Asymptotic symbol	Relational operator
θ	$=$
O	$\leq$
o	$<$
Ω	$\geq$
ω	$>$

Asymptotic Notations

- Which ones are used more often?

  $\theta - O - o - \Omega - \omega$

  - O : oldest notation

  - $f(n) \in O(\ldots)$ ✔

    $f(n) \in o(\ldots)$?

  - o : more information than O
    ω : more information than Ω

  - $f(n) \in O(\ldots)$ ✔

    $f(n) \in \theta(\ldots)$?

  - Ω and ω : in lower bound theory

Asymptotic Notations

- One parameter
  - $f(n) \in O(g(n))$ iff there exist positive constants c and n_0 such that $f(n) \leq c * g(n)$, whenever $n \geq n_0$
- Two parameters
  - $f(n,m) \in O(g(n,m))$ iff there exist positive constants c and n_0 such that $f(n,m) \leq c * g(n,m)$, whenever $n,m \geq n_0$
- So on

Asymptotic Notations

- Limit analysis:
  - Take the limit of the ratio of the two functions

$$\lim_{n\to\infty}\frac{f(n)}{g(n)}=\begin{cases}0 \implies f(n)\in o\big(g(n)\big)\\ c \implies f(n)\in \theta\big(g(n)\big)\\ \infty \implies f(n)\in \omega\big(g(n)\big)\\ 1 \implies f(n)\in \sim\big(g(n)\big)\end{cases}$$

strongly asymptotic to

- Convention: $f(n)\sim g(n)$
- Constants important
  $\sim(n^2)\neq\sim(5n^2)$

Asymptotic Notations

- Limit analysis:
  - L'hopital's rule

    Let $f(x)$ and $g(x)$ be functions that are differentiable for sufficiently large real numbers x. If $\lim_{x\to\infty} f(x) = \infty\ (or\ 0)$ and $\lim_{x\to\infty} g(x) = \infty\ (or\ 0)$, then

    $$\lim_{x\to\infty} \frac{f(x)}{g(x)} = \lim_{x\to\infty} \frac{f\prime(x)}{g\prime(x)}$$

  - Stirling's formula:

    $n! \cong \sqrt{2\pi n}\ \left(\frac{n}{e}\right)^n$, for large values of n

Asymptotic Notations

- Properties

  - Let f$(n), g(n) \in \mathcal{F}$

    - $f(n) \in O\big(g(n)\big) \Leftrightarrow g(n) \in \Omega\big(f(n)\big)$

      Example:

$$n \in O(n^2) \Leftrightarrow n^2 \in \Omega(n)$$

$$n^2 \in \{n, 5n, n\log n\,, n^2, n^2 + \log n\,, n^3, 2^n, n!, \dots\}$$

$$n \in \{n^2, 5n^2, n, n/2\,, \log n\,, \log n + 2n, n\log n\,, \dots\}$$

$\Longrightarrow$: implies
$\Leftrightarrow$: iff
$\subseteq$: subset
$\subset$: proper subset

Asymptotic Notations

- Properties

  - Let $f(n), g(n) \in \mathcal{F}$

    - $f(n) \in \theta(g(n)) \Leftrightarrow g(n) \in \theta(f(n))$

      Example:

$$5n^2 \in \theta(n^2) \Leftrightarrow n^2 \in \theta(5n^2)$$

$$n^2 \in \{n^2, 5n^2, n^2 + \log n, n^2 - \sqrt{n}/\log n, \dots\}$$

$$5n^2 \in \{n^2, 5n^2, n^2 + \log n, n^2 - \sqrt{n}/\log n, \dots\}$$

$\Rightarrow$: implies
$\Leftrightarrow$: iff
$\subseteq$: subset
$\subset$: proper subset

Asymptotic Notations

- Properties

  - Let $f(n), g(n) \in \mathcal{F}$

    - $f(n) \in \theta(g(n)) \Leftrightarrow f(n) \in O(g(n))\ and\ f(n) \in \Omega(g(n))$

      Example:

$$n^2 \in \theta(5n^2 + n) \Leftrightarrow n^2 \in O(5n^2 + n)\ and\ n^2 \in \Omega(5n^2 + n)$$

$$n^2 \in \{n^2, 5n^2, n^3, n^2 + \log n, 2^n, n!\}$$

$$n^2 \in \{n^2, 5n^2, n, n/2, \log n, \log n + 2n, n \log n, \dots\}$$

$$n^2 \in \{n^2, 5n^2, 5n^2 + n, n^2 + \log n, n^2 - \sqrt{n}/\log n, \dots\}$$

$\Longrightarrow$: implies
$\Leftrightarrow$: iff
$\subseteq$: subset
$\subset$: proper subset

Asymptotic Notations

- Properties
  - Let f$(n), g(n) \in \mathcal{F}$
    - $f(n) \in o\big(g(n)\big) \Leftrightarrow g(n) \in \omega\big(f(n)\big)$

      Example:

      $n^2 \in o(n^3) \Leftrightarrow n^3 \in \omega(n^2)$

$\Longrightarrow$: implies
$\Leftrightarrow$: iff
$\subseteq$: subset
$\subset$: proper subset

Asymptotic Notations

- Properties

  - Let f$(n), g(n) \in \mathcal{F}$

    - $f(n) \in o\big(g(n)\big) \Longrightarrow f(n) \in O\big(g(n)\big)$

      Example:

      $n^2 \in o(n^3) \Longrightarrow n^2 \in O(n^3)$

$\Longrightarrow$: implies
$\Leftrightarrow$: iff
$\subseteq$: subset
$\subset$: proper subset

Asymptotic Notations

- Properties

  - Let f$(n), g(n) \in \mathcal{F}$

    - $f(n) \in \omega\big(g(n)\big) \Longrightarrow f(n) \in \Omega\big(g(n)\big)$

      Example:

      $n^3 \in \omega(n^2) \Longrightarrow n^3 \in \Omega(n^2)$

$\Longrightarrow$: implies
$\Leftrightarrow$: iff
$\subseteq$: subset
$\subset$: proper subset

Asymptotic Notations

- Properties

  - Let f$(n), g(n) \in \mathcal{F}$

    - $\theta\big(f(n)\big) = \theta\big(c * f(n)\big)\,,\ c > 0$

      Example:

      $$\theta(n^2) = \theta(5 * n^2)$$

$\Longrightarrow$: implies
$\Leftrightarrow$: iff
$\subseteq$: subset
$\subset$: proper subset

Asymptotic Notations

- Properties

  - Let f$(n), g(n) \in \mathcal{F}$

    - $f(n) \in O\big(g(n)\big) \Leftrightarrow O\big(f(n)\big) \subseteq O\big(g(n)\big)$

      Example:

$$n \in O(n^2) \Leftrightarrow O(n) \subseteq O(n^2)$$

$$\{n, 5n, 1, \log n\,, \dots\} \subseteq \{n, 5n, 1, \log n\,, n\log n\,, n^2, n^2 + \log n\,, \dots\}$$

$$n \in \{n, 5n, 1, \log n\,, n\log n\,, n^2, n^2 + \log n\,, \dots\}$$

$\Longrightarrow$: implies
$\Leftrightarrow$: iff
$\subseteq$: subset
$\subset$: proper subset

Asymptotic Notations

- Properties
  - Let f$(n), g(n) \in \mathcal{F}$
    - $f(n) \in o\big(g(n)\big) \Leftrightarrow O\big(f(n)\big) \subset O\big(g(n)\big)$

      Example:

      $n \in o(n^2) \Leftrightarrow O(n) \subset O(n^2)$

$\Longrightarrow$: implies
$\Leftrightarrow$: iff
$\subseteq$: subset
$\subset$: proper subset

Asymptotic Notations

- Properties
  - Let f$(n), g(n) \in \mathcal{F}$
    - $f(n) \in \sim(g(n)) \Leftrightarrow f(n) \in \theta(g(n))$

      Example:

      $$n \in \sim(n + \log n) \Longrightarrow n \in \theta(n + \log n)$$

$\Longrightarrow$: implies
$\Leftrightarrow$: iff
$\subseteq$: subset
$\subset$: proper subset

Asymptotic Notations

- Properties

  - Let f$(n), g(n) \in \mathcal{F}$

    - $O\big(f(n)\big) = O\big(g(n)\big) \Leftrightarrow \theta\big(f(n)\big) = \theta\big(g(n)\big) \Leftrightarrow \Omega\big(f(n)\big) = \Omega\big(g(n)\big)$

      Example:

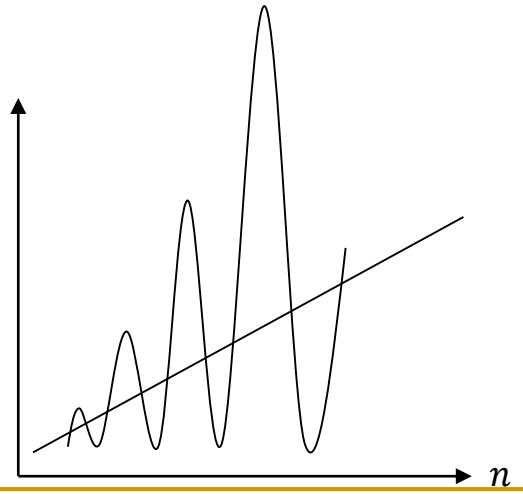
      $$O(5n^2) = O(n^2) \Leftrightarrow \theta(5n^2) = \theta(n^2) \Leftrightarrow \Omega(5n^2) = \Omega(n^2)$$

$\Longrightarrow$: implies
$\Leftrightarrow$: iff
$\subseteq$: subset
$\subset$: proper subset

Asymptotic Notations

- Definitions
  - Let f$(n), g(n) \in \mathcal{F}$
    - $f(n)$ and $g(n)$ have the same order if $f(n) \in \theta\big(g(n)\big)$ [or, $g(n) \in \theta\big(f(n)\big)$]
    - $f(n)$ has smaller order than $g(n)$ if $f(n) \in o\big(g(n)\big)$ [or, $O\big(f(n)\big) \subset O\big(g(n)\big)$]
    - $f(n)$ and $g(n)$ are not comparable if $f(n) \notin O\big(g(n)\big)$ and g$(n) \notin O\big(f(n)\big)$

Any examples?



Asymptotic Notations

Infinitely many functions with the same growth rate



single class

asymptotic notation

Example:

$$n^2, 5n^2, n^2 + \log n, \ldots \quad \Rightarrow \quad \theta(n^2)$$

- Still, infinitely many classes

Example:

$$\theta(n^2), \theta(n^{2.1}), \theta(n^{2.2}), \ldots$$

Asymptotic Notations

- $O(1)$ Constant order
  - Best possible complexity
  - Mostly occurs when analyzing $B(n)$

    Example: Binary search $\rightarrow B(n) \in O(1)$
  - Rarely occurs

    Example: Quick sort $\rightarrow B(n) \in O(n * \log n)$

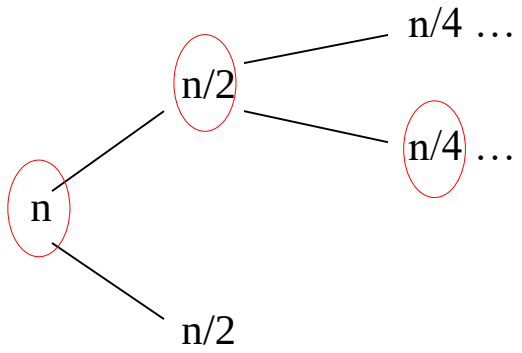
Asymptotic Notations

- $O(\log\log n)$, $O\left((\log\log n)^{1.5}\right)$, …
  - Grows very very slowly

    Example: $\log_e \log_e 1{,}000{,}000 = 2.6$

Asymptotic Notations

- $O(\log n)$ Logarithmic order



  - Grows very slowly

    Example: $\log_e 1{,}000{,}000 = 13.8$

Asymptotic Notations

- $O(\log n)$ Logarithmic order
  - Base of the logarithm is irrelevant

Important

$$\log_a n = \underbrace{\log_a b}_{c} * \log_b n, \text{ for } a, b > 1$$

same growth rate

$O(\log n), \theta(\log n), \ldots$ ✓

$O(\log_a n), \theta(\log_a n), \ldots$ ✗

Asymptotic Notations

- $O(n^a)$, $0 < a < 1$
  - Most frequent : $\sqrt{n}$
  - Grows faster than logarithmic order

    Example: $n^{0.01}$ & $\log n$

$$\lim_{n\to\infty} \frac{n^{0.01}}{\log n} = \lim_{n\to\infty} \frac{0.01 * n^{-0.99}}{1/n} = \lim_{n\to\infty} 0.01 * n^{0.01} = \infty$$

$$\Rightarrow n^{0.01} \in \omega(\log n)$$

Asymptotic Notations

- $O(n)$ Linear order
  - Another frequent order
  - Example:

    Algorithms that scan a list of size n

Asymptotic Notations

- $O(n * \log n)$
  - Another frequent order
  - Example:

    Divide and conquer algorithms

Asymptotic Notations

- $O(n^2)$ Quadratic order
  - Example:

    Algorithms with two nested loops

- $O(n^3)$ Cubic order
  - Example:

    Algorithms with three nested loops

Asymptotic Notations

- $O(a^n)$, $a > 1$ Exponential order
  - Most frequent : 2^n, 3^n

    $O(2^n)$: generating all subsets of an n-element list
  - Grows faster than any n^a
  - Grows very fast

    Example: Towers of Hanoi → $O(2^n)$

    Computer executing one trillion (10^{12}) operations per second

    input: 100 disks

    ⇒ 2^{100} → $4*10^{10}$ years

Asymptotic Notations

- $O(n * a^n)$, $a > 1$

- $O(n!)$ Factorial order
  - Grows faster than any exponential function
  - $O(n!)$

    → generating all permutations of an n-element list

- $O\left(a^{n^b}\right)$, $a, b > 1$

Asymptotic Notations

- Several others
  - $O(\log^2 n)$
  - $O(n^2 * \log^{1.5} n)$
  - …
  - Example:

    In detailed analysis $\rightarrow O(n^2 + \log^2 n + n * \sqrt{n})$

Asymptotic Notations

- Some terminology
  - $O(\log^a n)$, $a > 1$ $\rightarrow$ polylogarithmic
  - $o(\log n)$ $\rightarrow$ sublogarithmic
  - $o(n)$ $\rightarrow$ sublinear

Polynomial vs Exponential Algorithms

- An algorithm whose complexity is $O(n^a)$, where $a > 1$, is called a polynomial time algorithm

- An algorithm whose complexity is $\Omega(a^n)$, where $a > 1$, is called an exponential time algorithm

This distinction (polynomial time vs exponential time) is the most important issue for running of algorithms in practice

Polynomial vs Exponential Algorithms

- Order scale

$$O(1) \subset O(\log\log n) \subset O(\log n) \subset O(n^a) \subset O(n) \subset O(n\log n) \subset O(n^2) \subset O(n^3) \subset O(a^n) \subset O(na^n) \subset O(n!) \subset O\left(a^{n^b}\right)$$

polynomial algorithms

exponential algorithms

- Feasible algorithms
- Problems they solve are easy problems

- Infeasible algorithms
- Problems they solve are/may be hard problems

Polynomial algorithm → can execute in a reasonable amount of time
Exponential algorithm → can not